

Practical 4: Implementation and complexity analysis of algorithms based on dynamic programming. Floyd–Warshall Algorithm (All-Pairs Shortest Paths)

Floyd-Warshall Algorithm

Objective

- Unlike single-source algorithms like Dijkstra or Bellman-Ford, it determines the optimal distance between all possible start and end nodes simultaneously.
- It can correctly compute shortest paths in graphs containing negative edge weights, provided the graph contains no negative cycles.
- It systematically improves path estimates by considering each vertex as a potential intermediate pivot to find shorter routes between other nodes.

Theory

The Floyd-Warshall algorithm is a graph algorithm that is deployed to find the shortest path between all the vertices present in a weighted graph. This algorithm is different from other shortest path algorithms; to describe it simply, this algorithm uses each vertex in the graph as a pivot to check if it provides the shortest way to travel from one point to another.

There are mainly four steps in the algorithm:

- Construct an adjacency matrix A with all the costs of edges present in the graph. If there is no path between two vertices, mark the value as ∞
- Derive another adjacency matrix A_1 from A by keeping the first row and the first column of the original adjacency matrix unchanged. For the remaining entries, denoted by $A_1[i, j]$, update them according to the following rule:

$$A_1[i, j] = \begin{cases} A[i, 1] + A[1, j], & \text{if } A[i, j] > A[i, 1] + A[1, j], \\ A[i, j], & \text{otherwise.} \end{cases}$$

In this step, the pivot vertex is the first vertex, i.e., $k = 1$.

- Repeat Step 2 for all the vertices in the graph by changing the k value for every pivot vertex until the final matrix is achieved.
- The final adjacency matrix obtained is the final solution with all the shortest paths.

Algorithm

Algorithm 1 Quick Sort

```
1: procedure FLOYDWARSHALL( $W, n$ )
2:    $D \leftarrow W$ 
3:   for  $k \leftarrow 0$  to  $n - 1$  do
4:     for  $i \leftarrow 0$  to  $n - 1$  do
5:       for  $j \leftarrow 0$  to  $n - 1$  do
6:         if  $D[i][k] + D[k][j] < D[i][j]$  then
7:            $D[i][j] \leftarrow D[i][k] + D[k][j]$ 
8:         end if
9:       end for
10:    end for
11:  end for
12:  return  $D$ 
13: end procedure
```

Python Code

```
1 def Floyd_warsall(w, n):
2     D = [row[:] for row in w]
3
4     for k in range(n):
5         for i in range(n):
6             for j in range(n):
7                 if D[i][k] + D[k][j] < D[i][j]:
8                     D[i][j] = D[i][k] + D[k][j]
9
10    return D
```

Output

```
1 INF = float('inf')
2 n = 4
3 W = [
4     [0, INF, 3, INF],
5     [2, 0, INF, INF],
6     [INF, 7, 0, 1],
7     [6, INF, INF, 0]
8 ]
9
10 d = Floyd_warsall(W, n)
11 for i in range(n):
12     for j in range(n):
13         print(d[i][j], end='\t')
14     print()
15
16 0      10      3      4
17 2      0      5      6
18 7      7      0      1
19 6      16      9      0
```

Complexity Analysis

Time Complexity:

- $O(n^3)$

Space Complexity:

- $O(n^2)$

0/1 Knapsack Problem

Objective

- The primary goal is to select a subset of items that yields the highest possible profit or value. Mathematically, the objective is to maximize: $\sum_{i=1}^n p_i x_i$
- The total weight of all chosen items must never exceed the maximum carrying capacity of the knapsack.
- Unlike the fractional knapsack problem, an item cannot be broken into smaller pieces. The item must either be taken entirely or left completely behind.

Theory

Given two arrays, $val[]$ and $wt[]$, where each element represents the value and weight of an item respectively, also given an integer W representing the maximum capacity of the knapsack (the total weight it can hold). Put the items into the knapsack such that the sum of values associated with them is the maximum possible, without exceeding the capacity W .

Algorithm

Algorithm 2 Binary Knapsack (0/1 Knapsack)

```
1: function BINARYKNAPSACK(weights, values, capacity)
2:    $n \leftarrow \text{length}(\textit{weights})$ 
3:   Create a table  $dp[0 \dots n][0 \dots \textit{capacity}]$ 
4:   Initialize all elements of  $dp$  to 0
5:   for  $i \leftarrow 1$  to  $n$  do
6:     for  $w \leftarrow 0$  to  $\textit{capacity}$  do
7:        $dp[i][w] \leftarrow dp[i-1][w]$ 
8:       if  $\textit{weights}[i-1] \leq w$  then
9:          $dp[i][w] \leftarrow \max(dp[i][w], dp[i-1][w - \textit{weights}[i-1]] + \textit{values}[i-1])$ 
10:      end if
11:    end for
12:  end for
13:  return  $dp[n][\textit{capacity}]$ 
14: end function
```

Python Code

```
1 def binary_knapsack(weights, values, capacity):
2     n = len(weights)
3     dp = [[0] * (capacity + 1) for _ in range(n + 1)]
4
5     for i in range(1, n + 1):
6         for w in range(capacity + 1):
7             dp[i][w] = dp[i - 1][w]
8             if weights[i - 1] <= w:
9                 dp[i][w] = max(dp[i][w], dp[i - 1][w - weights[i - 1]] +
10                               values[i - 1])
11
12     return dp[n][capacity]
```

Output

```
1 weights = [2, 3, 4, 5]
2 values = [3, 4, 5, 6]
3 capacity = 5
4
5 print("The maximum profit is : ", binary_knapsack(weights, values, capacity))
6
7 The maximum profit is : 7
```

Complexity Analysis

Time Complexity:

- $O(n * M)$

Space Complexity:

- $O(n * M)$

Discussion

The Floyd-Warshall and 0/1 Knapsack algorithms use dynamic programming to solve problems efficiently by storing the results of smaller subproblems. The Floyd-Warshall algorithm finds the shortest paths between all pairs of vertices by updating the distance matrix using each vertex as an intermediate point. It is simple to implement and works correctly for graphs with negative edge weights if there are no negative cycles. The 0/1 Knapsack algorithm finds the maximum value that can be obtained without exceeding the given capacity by building a dynamic programming table. Both algorithms produce optimal results but require additional memory for storing intermediate values.

Conclusion

In this practical, the Floyd-Warshall and 0/1 Knapsack algorithms were successfully implemented and tested. The outputs obtained were correct and matched the expected results. This practical showed how dynamic programming improves efficiency by reusing previously computed results, making it a useful technique for solving optimization and graph-related problems.